An Autonomous Institution, Affiliated to Visvesvaraya Technological University, Belagavi.
Approved by AICTE, New Delhi | Recognized by UGC with 2(f) & 12(B) status | Accredited by NBA and NAAC

DEPARTMENT OF VLSI DESIGN AND TECHNOLOGY (ELECTRONICS ENGINEERING)

PROJECT REPORT

on

"Design of a Fingerprint-Based Payment System and Its Implementation on FPGA"

Academic Year 2025-26

Submitted by

CHANNAYYA V METI	:	1MJ23VL004
JEEVAN K HOSAKERI	:	1MJ23VL011
CHETAN TORGALMATH	:	1MJ23VL005
NARENDAR T RATHOR	:	1MJ24VL406

Under the guidance of

DR. SHENBHAGA LAKSHMI
Professor, Department of Electronics and Communication Engineering

MVJ College of Engineering, Bengaluru

CERTIFICATE

This is to certify that the project work titled "**Design of a Fingerprint-Based Payment System and Its Implementation on FPGA**" is a bonafide work carried out by **Channayya V Meti** (1MJ23VL004), **Jeevan K Hosakeri** (1MJ23VL011), **Chetan Torgalmath** (1MJ23VL005) and **Narendar T Rathor** (1MJ24VL406), students of the Department of VLSI Design and Technology (Electronics Engineering), MVJ College of Engineering, Bengaluru, during the academic year 2025-26.

The report fulfils the academic requirements for project work. All the suggestions and corrections given during the review have been included in the report.

Dr. Shenbhaga Lakshmi Project Guide Professor, Dept. of ECE	**Head of the Department** Dept. of VLSI Design and Technology	**Principal** MVJ College of Engineering

Examiners

Name and signature:

1. _____ 2. _____

DECLARATION

We, the undersigned students of the Department of VLSI Design and Technology (Electronics Engineering), MVJ College of Engineering, Bengaluru, declare that the project work titled "**Design of a Fingerprint-Based Payment System and Its Implementation on FPGA**" is the result of our own work, carried out under the guidance of Dr. Shenbhaga Lakshmi, Professor, Department of Electronics and Communication Engineering.

Wherever we have used the work of others, we have given the source in the references. To the best of our knowledge, this work has not been submitted to any other institution for any degree or diploma.

Channayya V Meti	1MJ23VL004	_____
Jeevan K Hosakeri	1MJ23VL011	_____
Chetan Torgalmath	1MJ23VL005	_____
Narendar T Rathor	1MJ24VL406	_____

Place: Bengaluru

Date:

ACKNOWLEDGEMENT

We would like to thank everyone who helped us complete this project.

First of all, we thank our guide, Dr. Shenbhaga Lakshmi, Professor, Department of Electronics and Communication Engineering, for her guidance, patience and encouragement through every stage of the work. Her questions at the reviews made us look again at parts of the design we had thought were finished.

We thank the Department of VLSI Design and Technology and all its faculty members for the teaching that this project builds on, and for giving us access to the FPGA board and the laboratory when we needed them. We are also grateful to the management and the Principal of MVJ College of Engineering, Bengaluru, for providing the environment and the resources to carry out the work.

We thank the authors of the papers listed in the references. Their work gave us a clear picture of where fingerprint payment systems stand and where the problems are.

Finally, we thank our families and friends for their support and for putting up with the long hours.

Channayya V Meti
Jeevan K Hosakeri
Chetan Torgalmath
Narendar T Rathor

ABSTRACT

Digital payments are convenient, but they are also an easy target for fraud, and much of the weakness lies in the things we use to prove who we are: PINs, passwords, one-time passwords and cards. This project studies fingerprint recognition as a replacement for those methods and builds a working payment authentication core on an FPGA.

The report has two parts. The first reviews published work on fingerprint-based payment systems: how willing users are to adopt them, the security and accuracy problems that remain, and three system architectures (database-centric, microcontroller-based and FPGA-based). The second part describes our own design. The system has two processes. During registration, a fingerprint template is captured and stored against an account. During a transaction, a live template is compared with the stored ones, and the payment is approved or declined.

We wrote the core in Verilog. It stores up to eight 128-bit templates, searches them with an XOR and bit-count (Hamming distance) comparison under a four-state finite state machine, and approves a transaction when the distance is within a set threshold. We verified it with a testbench that covers enrollment, an exact match, an approximate match and a non-match, and examined the transaction-level behaviour in a waveform simulation. The design was then demonstrated on an Artix-7 FPGA board with an optical fingerprint sensor.

The results show that the matching and authorisation decision can be made inside the FPGA without an operating system, which suits offline point-of-sale use. The report also sets out the limits of the prototype and the work we would do next.

Keywords: fingerprint authentication, biometric payment, FPGA, Verilog HDL, template matching, Hamming distance, Artix-7.

TABLE OF CONTENTS

Chapter 1: Introduction 1
1.1 Background 1
1.2 Biometric Authentication 1
1.3 Why an FPGA 1
1.4 Motivation 2
1.5 Scope of the Project 2
1.6 Organisation of the Report 2

Chapter 2: Literature Review 3
2.1 Overview 3
2.2 User Adoption and Perception 3
2.3 Security and Accuracy of Fingerprint Systems 3
2.4 System Architectures 4
2.5 Biometric Processing and Storage 5
2.6 Comparison of the Architectures 6
2.7 Summary of the Papers Reviewed 6
2.8 Observations from the Review 7

Chapter 3: Problem Statement, Objectives and Methodology 8
3.1 Problem Statement 8
3.2 Objectives 8
3.3 Methodology 8

Chapter 4: Existing and Proposed System 9
4.1 Existing System 9
4.2 Proposed System 9
4.3 Registration and Transaction Processes 9

Chapter 5: System Design 11
5.1 Block Diagram 11
5.2 Flow Chart 11
5.3 Design Decisions 12

Chapter 6: Hardware and Software Requirements 13
6.1 Hardware Components 13
6.2 The FPGA Board 13
6.3 Software and Tools 13

Chapter 7: Implementation 14
7.1 Overview of the Verilog Design 14
7.2 Module Interface 14
7.3 Internal Storage 14

7.4 Finite State Machine 15
7.5 Enrollment Operation 15
7.6 Matching Operation 16
7.7 Testbench 16

Chapter 8: Results and Discussion 18
8.1 Simulation Setup 18
8.2 Transaction-Level Waveform 18
8.3 Testbench Outcomes 19
8.4 Hardware Implementation 19
8.5 Discussion 20

Chapter 9: Advantages, Applications, Limitations and Future Scope 21
9.1 Advantages 21
9.2 Applications 21
9.3 Limitations 21
9.4 Future Scope 22

Chapter 10: Conclusion 23
10.1 Summary 23
10.2 Contributions of Team Members 23

References 24

Appendix A: Verilog Source Code of the Fingerprint Payment Core 25

Appendix B: Testbench Source Code 29

LIST OF FIGURES

Figure 2.1: Block diagram of the FPGA-based payment system (adapted from Rong and Liu [6]). 5

Figure 5.1: Functional block diagram of the proposed fingerprint payment system. 11

Figure 5.2: Flow chart of the enrollment and transaction phases. 12

Figure 7.1: State diagram of the finite state machine. 15

Figure 8.1: Simulation waveform showing fp_id, fp_valid, amount_in, start_tx, success, fail and acc_balance. 18

Figure 8.2: Hardware implementation: Nexys A7 (Artix-7) board with the optical fingerprint sensor. 20

LIST OF TABLES

Table 2.1: Comparison of the three architectures found in the literature. 6

Table 2.2: Summary of the papers reviewed. 6

Table 4.1: Authentication methods in current payment systems and their weaknesses. 9

Table 6.1: Hardware components and their purpose. 13

Table 6.2: Software and tools used in the project. 13

Table 7.1: Parameters of the fingerprint_payment module. 14

Table 7.2: Ports of the fingerprint_payment module. 14

Table 7.3: States of the finite state machine. 15

Table 7.4: Test cases in the testbench and the Hamming distances they produce 17

Table 8.1: Values read from the waveform, with decimal equivalents. 18

Table 8.2: Sequence of events in the waveform (approximate times). 19

Table 10.1: Contribution of each team member. 23

LIST OF ABBREVIATIONS

Abbreviation	Full form
ANN	Artificial Neural Network
CNN	Convolutional Neural Network
EMV	Europay, Mastercard and Visa (chip card standard)
FAR	False Acceptance Rate
FPGA	Field-Programmable Gate Array
FRR	False Rejection Rate
FSM	Finite State Machine
HDL	Hardware Description Language
IC	Integrated Circuit
OTP	One-Time Password
PIN	Personal Identification Number
PLS-SEM	Partial Least Squares Structural Equation Modelling
POS	Point of Sale
PS/2	Personal System/2 (keyboard interface)
RTL	Register Transfer Level
UIDAI	Unique Identification Authority of India
UTAUT2	Unified Theory of Acceptance and Use of Technology 2
VGA	Video Graphics Array
VLSI	Very Large Scale Integration

CHAPTER 1
INTRODUCTION

1.1 Background

For a long time, everyday payments ran on cash and, later, on IC (Integrated Circuit) cards. Both carry the same trade-off between convenience and security. Cash is slow when exact change is needed and is awkward when the seller is offline. A card is quicker, but on its own it only proves that someone is holding the card, so a lost or stolen card can be used by anyone.

Digital and mobile payments tried to fix this by adding something the user knows: a password or a Personal Identification Number (PIN). These knowledge-based methods have problems of their own. They can be forgotten, watched over a shoulder, stolen or guessed. One-Time Passwords (OTPs) add a second step, but they can still be intercepted through SIM swapping or phishing. As digital payments have grown, so has financial fraud, and security is now the biggest concern with cashless transactions. In all of these methods most of the burden of keeping the credential safe falls on the user.

1.2 Biometric Authentication

Authentication usually rests on one of three things: something the user knows (a PIN), something the user has (a card), or something the user is. Biometric authentication uses the third. It identifies a person from physiological or behavioural characteristics such as the face, iris, voice or fingerprint.

Among these, the fingerprint is the most developed, the most widely accepted and one of the cheapest to deploy. A fingerprint is distinctive to each person and stays the same through life, which makes it a dependable way to recognise someone. Its pattern is described by minutiae, mainly ridge endings and bifurcations, which can be turned into a compact template for storage and comparison.

1.3 Why an FPGA

As biometric payments move from online platforms to physical point-of-sale (POS) terminals, there is a need for terminals that are cheap, secure and able to work on their own. A Field-Programmable Gate Array (FPGA) suits this need well. The logic for sensor control, template matching and transaction handling can be written once in a hardware description language and then runs directly as hardware. There is no operating system to attack, the circuit behaves the

same way every time, and independent blocks can work in parallel. For a POS device this can give a system that is both fast and hard to tamper with through software.

1.4 Motivation

The project started from two observations. First, the usual ways of authorising a payment depend on secrets that are easy to lose or steal. Second, most fingerprint payment systems in the literature either need a constant network connection to a central database or run on a general-purpose processor with an operating system. We wanted to see how much of the payment decision could be made inside a small, self-contained digital circuit. That is also the kind of design problem our VLSI coursework prepares us for.

1.5 Scope of the Project

The project covers four things:

- a study of existing fingerprint-based payment systems and their problems;
- the design of a template storage and matching core in Verilog;
- verification of that core with a testbench and a waveform simulation;
- a demonstration on an Artix-7 FPGA board with an AS608 optical fingerprint sensor.

Some things are outside the scope of this prototype: liveness detection (spoof detection), a complete bank or wallet back end, production-grade encryption of stored templates, and large-scale accuracy testing with many users. Chapter 9 comes back to these points.

1.6 Organisation of the Report

- **Chapter 2** reviews the literature on fingerprint payment systems.
- **Chapter 3** states the problem, the objectives and the method we followed.
- **Chapter 4** compares the existing and the proposed system.
- **Chapter 5** presents the system design: block diagram and flow chart.
- **Chapter 6** lists the hardware and software used.
- **Chapter 7** describes the Verilog implementation and the testbench.
- **Chapter 8** presents the simulation and hardware results.
- **Chapter 9** covers advantages, applications, limitations and future scope.
- **Chapter 10** concludes the report and records each member's contribution.

CHAPTER 2
LITERATURE REVIEW

2.1 Overview

We reviewed the published work in three parts. First, whether users want biometric payments at all. Second, the technical problems that stand in the way, mainly security and accuracy. Third, the system architectures that researchers have proposed, including the FPGA-based approach that our project follows. Table 2.2 at the end of the chapter lists the eight papers used.

2.2 User Adoption and Perception

Studies show that people are open to biometric payments mainly because they believe the method is safe and practical. Adoption is driven by the belief that the system will work well (Performance Expectancy) and by how easy it is to use (Effort Expectancy). Perceived risk has only a small negative effect, which suggests that users already regard biometrics as safer than conventional methods [2].

An experimental study by Ogbanufe and Kim [1] compared fingerprint authentication with traditional methods for e-payment. Subjects who used a fingerprint perceived far less security concern than those who used only a credit card, or a credit card with a PIN. That higher sense of security turned into greater trust in the online store. The same line of work found no significant difference in perceived convenience between a fingerprint and a credit card, which points to security, and not speed, as the main benefit users see. Liébana-Cabanillas et al. [2] examined the intention to use biometric m-payments with the UTAUT2 framework, combining Structural Equation Modelling (PLS-SEM) with Artificial Neural Networks (ANN) to find which factors matter most.

2.3 Security and Accuracy of Fingerprint Systems

Yang et al. [3] review the main technical issues. On the security side there are two threats. The first is sensor spoofing, where a fake fingerprint is presented to the scanner. The second is theft of the template database, and it is the more serious of the two. A password can be changed after a leak, but a fingerprint cannot, so a stolen template is a permanent breach. The usual countermeasures are template protection, such as cancelable biometrics or biometric cryptosystems, to secure what is stored, and liveness detection to catch fake fingers.

Accuracy is the other side. Modern matching algorithms are very accurate under good conditions, but they do worse when the input is poor or distorted. Matching of latent fingerprints, for example, is still far from satisfactory. The two issues also pull against each other. Strong template protection transforms or hides information, and that loss can lower matching accuracy and raise the false rejection rate. One study discussed in the review reported a False Rejection Rate (FRR) of 3.18% at 0% False Acceptance Rate (FAR) with the original features, but an FRR of 51.29% once high-security template protection was applied [3]. Any design therefore has to balance how well it protects the template against how often it wrongly turns away a genuine user.

2.4 System Architectures

Three architectures appear in the literature.

2.4.1 Database-centric online model

The payment terminal captures the user's fingerprint and an identifier, such as an Aadhaar number, and sends an authentication request over the network to a large central database, for example the UIDAI database, through a bank. The core step is a 1:1 online match that authorises the transaction. Security is high because the data sits in a controlled central system, but the model depends on a stable internet connection and cannot work offline.

2.4.2 Microcontroller-based or hybrid model

A low-cost microcontroller or single-board computer, typically an ESP32 or a Raspberry Pi, acts as the brain of the POS terminal. The software is usually written in Python, with libraries such as OpenCV for face recognition or adafruit_fingerprint for talking to the sensor. Data is protected with software methods such as SHA-256 hashing and is sent over HTTP or HTTPS to a central MySQL database. Because a server is needed for data management and the system runs on an operating system, the model is flexible but has a larger attack surface.

2.4.3 FPGA-based offline model

Rong and Liu [6] describe a payment system built purely on an FPGA. A finite state machine in hardware is the core. It drives a VGA display module, by generating the horizontal and vertical sync signals directly, and a PS/2 keyboard module. The FPGA talks to a fingerprint module over a serial port by sending commands and reading responses. The system is designed for offline use: it does a 1:N (one-to-many) search against the templates stored in the fingerprint module's own library, so no outside database is needed. Their prototype on a Xilinx Artix-7 board (Basys 3) supported opening an account, recharging and spending, with an average

payment time below one second and a FAR below 0.001% [6]. Figure 2.1 shows the block diagram used in that work.

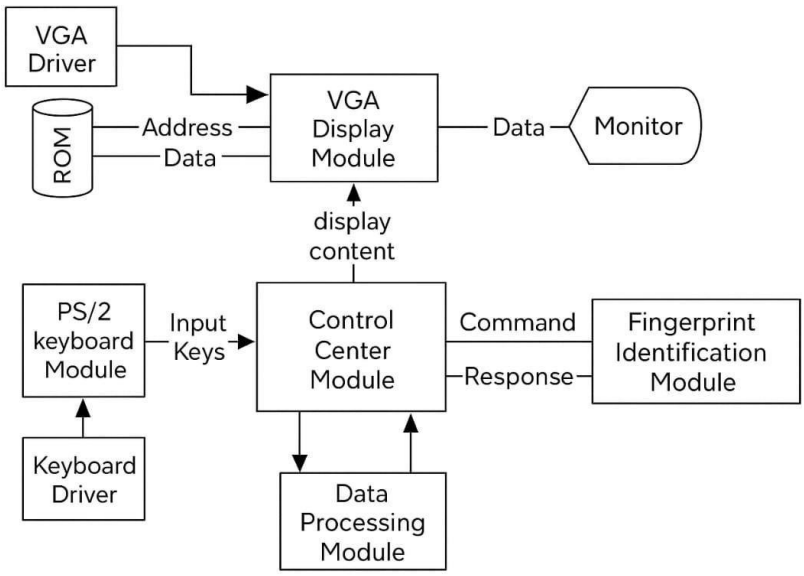


Figure 1. Block diagram of the payment system

Figure 2.1: Block diagram of the FPGA-based payment system (adapted from Rong and Liu [6]).

2.5 Biometric Processing and Storage

Across the architectures, the literature describes a common set of steps for handling the fingerprint data itself.

- **Enrollment.** The user normally places the finger twice. The processor in the sensor module combines the two images into a single good template, which is then stored.
- **Feature extraction.** A minutiae extraction algorithm processes the image to find ridge endings and bifurcations. Each minutia is kept as a small record with its type, X and Y coordinates and angle.
- **Template storage.** A database-centric system keeps the template in a secure central government database. A microcontroller system can store a SHA-256 hashed template in a private MySQL database. An FPGA system can keep the template in the fingerprint module's internal library, so that the whole device works as one self-contained unit.

The systems are evaluated in two ways. For technical performance, a physical prototype is built and tested; the FPGA system above was tested on a Basys 3 board and showed an average payment time under one second with a FAR below 0.001%. For user adoption, surveys and statistical models such as UTAUT2, PLS-SEM and ANN are used to find which factors, such as performance, effort and perceived risk, most influence a user's intention to adopt the technology.

2.6 Comparison of the Architectures

Table 2.1: Comparison of the three architectures found in the literature.

Aspect	Database-centric	Microcontroller-based	FPGA-based
Where matching happens	Central server (for example UIDAI), reached through a bank	Software on an ESP32 or Raspberry Pi	In hardware, as a 1:N search against the module's template library
Network need	Constant internet connection	Server connection for data management	Works offline
Template storage	Central government database	SHA-256 hashed, in a private MySQL database	Inside the fingerprint module's internal library
Main strength	High security, very large existing database	Low cost and flexible, software-level encryption	Hardware-only logic, fast, offline
Main weakness	Cannot work offline	Software and operating system enlarge the attack surface	Needs hardware design effort

2.7 Summary of the Papers Reviewed

Table 2.2: Summary of the papers reviewed.

Ref.	Authors and year	Title	Inference
[1]	Ogbanufe and Kim, 2018	Comparing fingerprint-based biometrics authentication versus traditional authentication methods for e-payment	Fingerprint users perceived less security concern than card or card-plus-PIN users, and trusted the online store more.
[2]	Liébana-Cabanillas et al., 2024	Biometric m-payment systems: a multi-analytical approach to determining use intention	UTAUT2 with PLS-SEM and ANN; performance expectancy and effort expectancy drive the intention to use.
[3]	Yang et al., 2019	Security and accuracy of fingerprint-based biometrics: a review	Security and accuracy are the two key design aspects; strong template protection can reduce accuracy.
[4]	Nayak et al., 2024	Payment transaction using biometric authentication	Integrates biometrics into payment platforms to fight fraud and simplify authentication.
[5]	Lavadkar et al.	Fingerprint biometric based online cashless payment system	Online cashless payment model that uses Aadhaar-based biometric authentication to secure banking transactions.
[6]	Rong and Liu, 2016	A fingerprint payment system based on FPGA	Complete offline payment system on an Artix-7 FPGA with fingerprint module, VGA display and PS/2 keyboard; under 1 s per payment, FAR below 0.001%.
[7]	Logeswari et al., 2018	Fingerprint based online payment system	Part of the survey of fingerprint-based online payment systems.

Ref.	Authors and year	Title	Inference
[8]	Banumathi et al., 2024	Biometric based payment system	Biometric payment system designed for streamlined transactions with security as the priority.

2.8 Observations from the Review

- Users already trust fingerprint payments more than cards or PINs, and they value the security more than the speed.
- Template theft and sensor spoofing are the main security risks, and protecting a template can cost accuracy.
- Database-centric systems are secure but need a network. Microcontroller systems depend on software and an operating system.
- An FPGA system keeps the payment and authentication logic in hardware, which removes many software-level attacks and allows offline use. This is the direction our project takes: a small, parameterised matching core that can be simulated and then tried out on a standard Artix-7 board.

CHAPTER 3
PROBLEM STATEMENT, OBJECTIVES AND METHODOLOGY

3.1 Problem Statement

To design and implement a secure cashless transaction system that uses fingerprint biometrics to overcome the weaknesses of PINs, passwords and OTPs. The system must identify the user accurately and quickly enough to authorise a payment, and it must prevent unauthorised access and fraudulent transactions.

3.2 Objectives

- To study existing fingerprint-based payment systems and identify their security and accuracy limits.
- To design a payment authentication architecture with a registration process and a transaction process.
- To implement template storage, matching and the transaction decision in Verilog, using a finite state machine.
- To verify the design with a testbench and a waveform simulation that cover exact, approximate and non-matching fingerprints.
- To demonstrate the design on an Artix-7 FPGA board with a fingerprint sensor.

3.3 Methodology

We worked through the project in the following order.

1. **Literature survey.** We studied eight papers on user adoption, security, accuracy and system architecture (Chapter 2).
2. **Architecture.** We drew the block diagram and the flow chart for the registration and transaction processes (Chapter 5).
3. **RTL design.** We wrote the template storage, matching and decision logic in Verilog as a finite state machine (Chapter 7).
4. **Verification.** We wrote a testbench with enrollment, exact-match, approximate-match and non-match cases and checked the waveforms (Chapters 7 and 8).
5. **Hardware implementation.** We set up the design on an Artix-7 board with the fingerprint sensor (Chapter 8).

6. **Documentation.** We collected the results and wrote this report.

CHAPTER 4
EXISTING AND PROPOSED SYSTEM

4.1 Existing System

Current transaction systems rely on methods that can be breached. Table 4.1 lists the common ones and where each of them fails.

Table 4.1: Authentication methods in current payment systems and their weaknesses.

Method	Weakness
PINs and passwords	Can be forgotten, stolen or brute-forced.
Magnetic stripe cards	Easily cloned.
EMV chip cards	Secure, but they still depend on a PIN.
One-Time Passwords (OTPs)	Vulnerable to SIM swapping and phishing.

What these methods have in common is that they put the burden of security on the user. That has proved to be a weakness in practice.

4.2 Proposed System

Our system builds a fingerprint scanner into the payment step. It works in four stages.

1. **Enrollment.** The user's fingerprint is registered, turned into a secure digital template and linked to an account.
2. **Authentication.** The user scans a fingerprint at the time of payment.
3. **Verification.** Features are extracted from the live scan and compared with the stored templates.
4. **Authorization.** The transaction is approved if the fingerprint matches and declined if it does not.

Compared with the existing methods, this gives three advantages:

- **Security.** A fingerprint is much harder to forge than a PIN or a card.
- **Convenience.** There is no PIN to remember and no card to carry.
- **Speed.** Authentication takes place the moment the finger is placed.

4.3 Registration and Transaction Processes

The system has two main processes. In the registration process, the fingerprint is captured, its features are extracted and the resulting template is stored against the user's account. In the

transaction process, a live scan is taken, converted to a template in the same way and matched against the stored templates. The outcome of that match decides whether the payment goes through. Chapter 5 shows both processes as a flow chart, and Chapter 7 describes how they are implemented in the FPGA.

CHAPTER 5
SYSTEM DESIGN

5.1 Block Diagram

Figure 5.1 shows the functional blocks of the proposed system. The fingerprint sensor captures the image. The feature extraction module turns it into a template of minutiae. The matching engine compares that template with the ones held in the template database. The transaction authorization block uses the result to approve or decline the payment, and the outcome is shown on the display or sent to the host system.

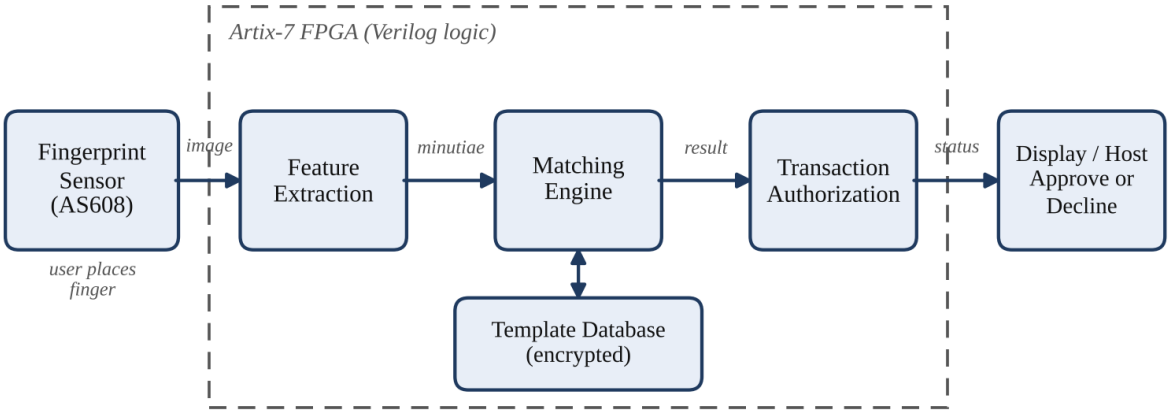


Figure 5.1: Functional block diagram of the proposed fingerprint payment system.

The key modules are:

- **Fingerprint sensor:** captures a high-resolution fingerprint image.
- **Feature extraction module:** identifies the minutiae points.
- **Database:** stores the fingerprint templates in encrypted form.
- **Matching engine:** compares the live scan with the stored templates.
- **Transaction authorization system:** approves transactions that have been verified.

5.2 Flow Chart

Figure 5.2 shows the flow of the two phases. In the enrollment phase the user places a finger on the sensor, the sensor captures an image, the image is converted to a digital template and the template is stored in the secure database, linked to the user's account. In the transaction phase the user places a finger on the scanner at the POS, the system captures a live sample and converts it to a template, and the template is compared with those in the database. If a match is

found, the payment is authorised and the amount is deducted from the linked account. If not, the payment is declined.

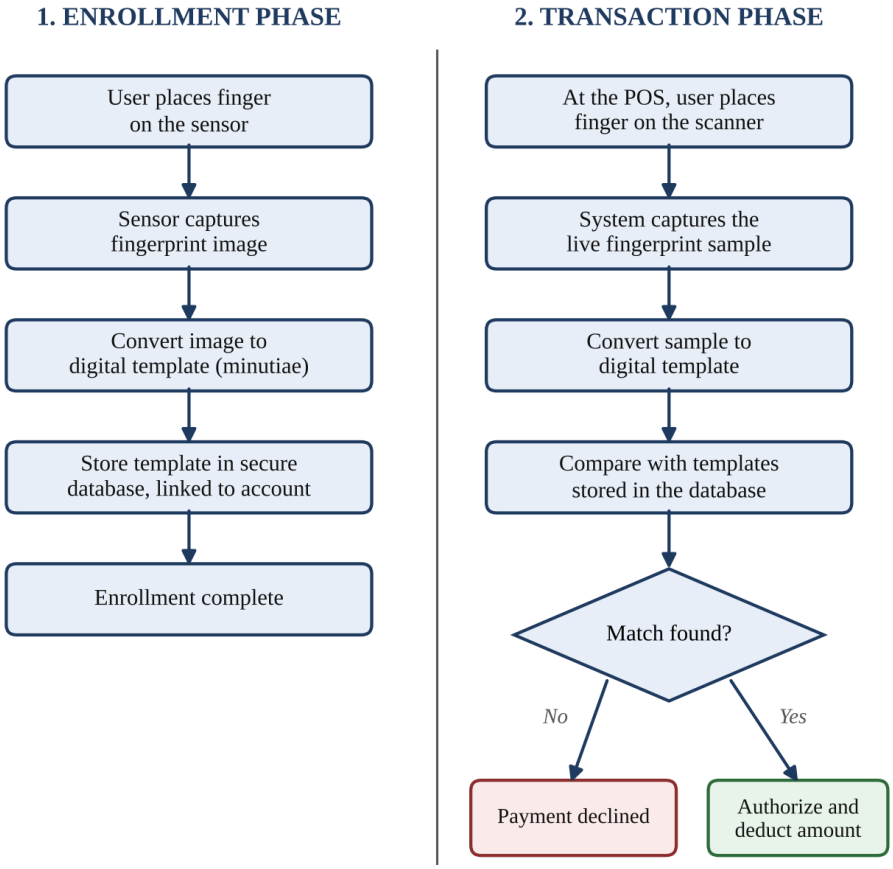


Figure 5.2: Flow chart of the enrollment and transaction phases.

5.3 Design Decisions

Several choices in the Verilog core follow from the aim of keeping the design small and easy to verify.

- **Fixed-length template.** Each fingerprint is represented by one 128-bit word. This keeps the storage and the comparison logic simple.
- **Eight template slots.** The templates are held in a small register file, so the whole library is inside the FPGA fabric.
- **Hamming-distance matching.** Two templates are compared by counting the bits in which they differ. A transaction is approved when that count is within a threshold, so small differences between two scans of the same finger are tolerated.
- **Finite state machine control.** A four-state machine controls enrollment and transactions, which makes the behaviour predictable and easy to follow in a waveform.

- **Serial bit counting.** The differing bits are counted one per clock cycle. This keeps the logic small at the cost of more clock cycles per comparison; Section 7.6 estimates that cost.

CHAPTER 6
HARDWARE AND SOFTWARE REQUIREMENTS

6.1 Hardware Components

Table 6.1: Hardware components and their purpose.

Component	Purpose
AS608 optical fingerprint sensor	Captures the fingerprint image of the user during registration and at the time of payment.
Artix-7 FPGA board	Acts as the main control unit. It performs fingerprint processing and matching, and controls communication with the other modules.
Host system (PC)	Shows messages such as "Place Finger", "Payment Successful" or "Access Denied", and can accept inputs such as the payment amount or an ID. It also acts as the external system that processes the payment after the FPGA has verified the user.

6.2 The FPGA Board

The prototype uses a Digilent Nexys A7 board, which carries an Artix-7 100T device (CSG324 package), as marked on the board in Figure 8.2. The board has an on-board 100 MHz clock, push buttons, switches, LEDs, a seven-segment display and Pmod headers. The fingerprint sensor connects to the board through jumper wires, and the board is powered and programmed through its USB port.

6.3 Software and Tools

Table 6.2: Software and tools used in the project.

Software	Purpose
Verilog HDL code	Implements the control logic in the FPGA.
Fingerprint matching algorithm	Verifies the identity of the user.
Database management software	Stores the templates and the payment records.
Encryption and security module	Protects the biometric and the financial data.
User interface	Displays the transaction and the user information.
Vivado simulator and waveform viewer	Used to simulate the design and inspect the signals (Chapter 8).

CHAPTER 7
IMPLEMENTATION

7.1 Overview of the Verilog Design

The core of the system is one module, `fingerprint_payment`. It stores enrolled templates, accepts a live template for a transaction, searches the stored templates and reports whether the transaction is approved or declined. The full source code is in Appendix A and the testbench is in Appendix B. The module is parameterised, as listed in Table 7.1.

Table 7.1: Parameters of the fingerprint_payment module.

Parameter	Default	Meaning
TEMPLATE_W	128	Width of one fingerprint template in bits.
SLOTS	8	Number of templates that can be stored.
THRESHOLD	8	Largest Hamming distance (number of differing bits) that still counts as a match. The testbench overrides it with 4.

7.2 Module Interface

Table 7.2: Ports of the fingerprint_payment module.

Port	Dir.	Description
clk	In	System clock.
rst_n	In	Active-low reset. It clears all stored templates and returns the state machine to IDLE.
enroll_valid	In	Request to store the template on enroll_template.
enroll_template	In	128-bit template to be enrolled.
enroll_ack	Out	Pulses high for one clock cycle after a template has been stored.
tx_valid	In	Request to start a transaction with the template on tx_template.
tx_template	In	128-bit live template to be checked.
tx_processing	Out	High while the search is running.
tx_approved	Out	Pulses high when a match is found.
tx_declined	Out	Pulses high when no stored template matches.
matched_slot	Out	Index of the stored template that matched.

7.3 Internal Storage

Enrolled templates live in a register file, `template_mem`, with one 128-bit entry per slot. A flag vector, `slot_used`, records which slots are occupied. For the search, the module keeps a

few working registers: `search_idx` (the slot being compared), `xor_val` (the XOR of the stored and the live template), `pop_idx` and `hamming_count` (used to count the differing bits), `pop_running` (a flag while counting) and `match_found` and `best_match_idx` (the result).

7.4 Finite State Machine

A four-state finite state machine controls the module. Figure 7.1 shows the states and the transitions, and Table 7.3 describes what happens in each state.

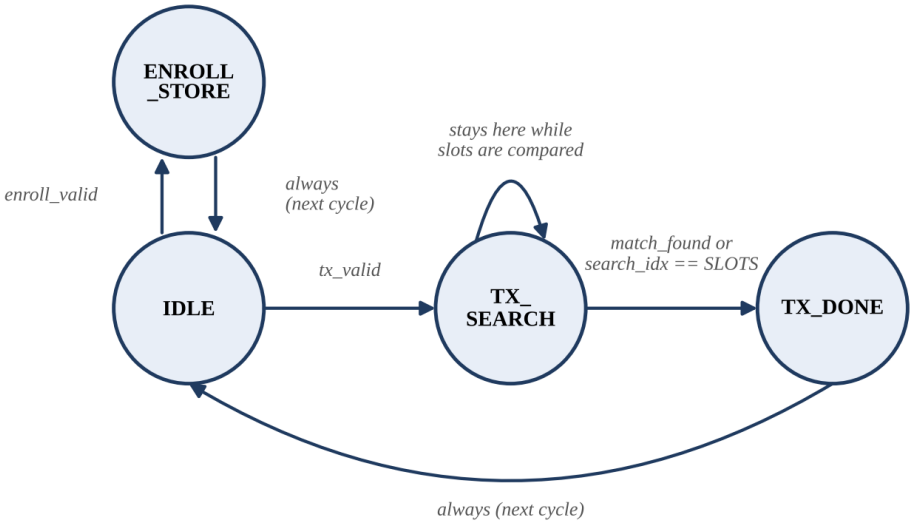


Figure 7.1: State diagram of the finite state machine.

Table 7.3: States of the finite state machine.

State	Behaviour
IDLE	Waits for a request. Working registers are cleared. A request to enroll moves the machine to ENROLL_STORE; a transaction request moves it to TX_SEARCH.
ENROLL_STORE	Stores the template in the first free slot, marks the slot as used and pulses enroll_ack. The machine returns to IDLE on the next clock.
TX_SEARCH	Compares the live template with each used slot in turn. The machine stays here until a slot matches or all slots have been checked.
TX_DONE	Raises tx_approved (with the matched slot number) or tx_declined, lowers tx_processing and returns to IDLE.

7.5 Enrollment Operation

When enroll_valid is high in the IDLE state, the machine moves to ENROLL_STORE. The module looks for the first slot whose flag is clear, writes the template into that slot, sets the flag and pulses enroll_ack. Templates are therefore stored in the order in which they are enrolled, and a reset clears all of them.

7.6 Matching Operation

When tx_valid is high in the IDLE state, the machine moves to TX_SEARCH. For each used slot the module does the following.

1. It takes the bitwise XOR of the stored template and the live template. Every bit that differs becomes a 1.
2. It counts the ones in the XOR result, one bit per clock cycle. The count is the Hamming distance between the two templates.
3. If the distance is less than or equal to THRESHOLD, the slot is a match and the search stops. Otherwise the module moves on to the next used slot.

Empty slots are skipped. If a match is found, or if all slots have been checked, the machine moves to TX_DONE and the transaction is approved or declined. Note that the search stops at the first slot that falls within the threshold; it does not look for the closest one.

Worked example. Take two templates that are identical except for the last four bits, where one ends in 0101 and the other in 0100. Their XOR has a single 1, so the Hamming distance is 1. With a threshold of 4 this is accepted. If the two templates were unrelated, the distance would be much larger and the transaction would be declined.

Choice of threshold. A low threshold makes the system strict and increases the chance that a genuine user is rejected. A high threshold makes it forgiving and increases the chance that a wrong finger is accepted. As a rough indication, two unrelated random 128-bit words have an expected distance of 64. The chance that such a pair differs in 4 bits or fewer is of the order of 10^-32, and for 8 bits or fewer of the order of 10^-27. Real fingerprint templates are not random, so these numbers are not a measured FAR; they only show that the thresholds used here sit far below the distance expected between unrelated templates.

Timing. Counting takes one clock cycle per bit, so one comparison needs about 128 cycles plus a few cycles of overhead. In the worst case, with all eight slots occupied and no match, the search takes roughly a thousand clock cycles. At the 100 MHz board clock that is about 10 microseconds. This is an estimate from the design, not a measured value, and it covers only the matching step. The time taken by the sensor to capture an image and by the serial link is far longer.

7.7 Testbench

The testbench `tb_fingerprint_payment` instantiates the module with TEMPLATE_W = 128, SLOTS = 8 and THRESHOLD = 4. It generates a clock with a 10 ns period, holds reset low for 20 ns and then applies the five test cases in Table 7.4. After each transaction it waits for tx_processing to fall and prints whether the transaction was approved or declined.

Table 7.4: Test cases in the testbench and the Hamming distances they produce.

No.	Test	Stimulus	Distance to slot 0 / slot 1	Expected result
1	Enroll slot 0	128'hA5A5...A5A5 (repeating A5)	-	enroll_ack pulse
2	Enroll slot 1	128'hFFFFFFFF00000000FFFFFFFF00000000	-	enroll_ack pulse
3	Transaction, exact match	Same value as slot 0	0 / 64	Approved, slot 0
4	Transaction, approximate match	A5A5...A5A4 (last hex digit changed, 1 bit differs)	1 / 63	Approved, slot 0
5	Transaction, non-match	128'h0123456789ABCDEF0123456789ABCDEF	48 / 80	Declined

The distances in the table were worked out from the hexadecimal values in the testbench. With a threshold of 4, only tests 3 and 4 fall inside the threshold for any enrolled slot, which is why they are expected to be approved and test 5 is expected to be declined.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Simulation Setup

The design was simulated with the Vivado simulator. The testbench in Appendix B checks enrollment and the three kinds of transaction. The transaction-level behaviour, that is the fingerprint ID, the amount and the account balance, was checked in the waveform window shown in Figure 8.1.

8.2 Transaction-Level Waveform

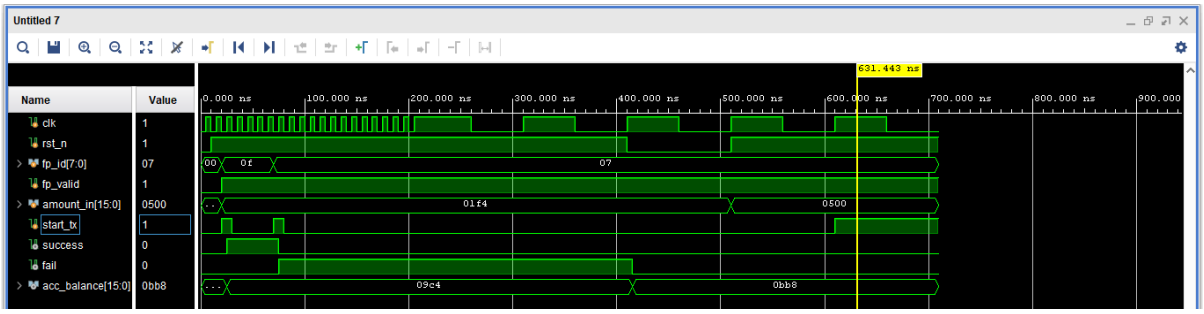


Figure 8.1: Simulation waveform showing fp_id, fp_valid, amount_in, start_tx, success, fail and acc_balance.

The signals in the waveform are the following. All values are shown in hexadecimal in the viewer; Table 8.1 gives the decimal equivalents.

- `clk` is the system clock; all operations are synchronised to it.
- `rst_n` is the active-low reset. It is high during normal operation.
- `fp_id` is the fingerprint ID that is presented to the system.
- `fp_valid` goes high to show that a valid fingerprint has been presented.
- `amount_in` is the amount requested.
- `start_tx` is a pulse that starts a transaction.
- `success` and `fail` report the outcome of a transaction.
- `acc_balance` is the account balance register.

Table 8.1: Values read from the waveform, with decimal equivalents.

Signal	Hex value	Decimal	When
fp_id	0F, then 07	15, then 7	0F from about 20 ns; 07 from about 70 ns onwards

Signal	Hex value	Decimal	When
amount_in	01F4	500	From about 20 ns until about 510 ns
amount_in	0500	1280	From about 510 ns to the end of the capture
acc_balance	09C4	2500	From about 25 ns until about 415 ns
acc_balance	0BB8	3000	From about 415 ns to the end of the capture

Table 8.2 follows the waveform in time order. The times are read from the waveform window and are approximate.

Table 8.2: Sequence of events in the waveform (approximate times).

Time	Event
0 to 10 ns	The clock is running and rst_n is released, so the system is active.
about 20 ns	A fingerprint ID (0F) is presented and fp_valid goes high. The first start_tx pulse begins a transaction for the amount 01F4 (500).
about 25 to 75 ns	success is high and fail is low: the transaction is approved. acc_balance shows 09C4 (2500) from this point.
about 75 ns	The ID changes to 07 and a second start_tx pulse is issued. fail goes high and stays high until about 415 ns. acc_balance does not change in this period, so the rejected request leaves the account untouched.
about 410 to 510 ns	rst_n is pulled low and released again. acc_balance returns to 0BB8 (3000) when the reset is applied.
about 510 ns	amount_in changes to 0500 (1280).
about 610 ns	start_tx goes high again with fp_id 07. The capture ends at about 700 ns before the outcome of this request is shown.

Reading the capture as a whole: after an approved transaction of 500 units the balance register holds 2500, and it goes back to 3000 when the reset is applied. Since 3000 minus 500 is 2500, this fits an account that starts at 3000 units and is debited by the approved amount. A rejected request, with fail high, does not change the balance.

8.3 Testbench Outcomes

For the testbench in Appendix B, the Hamming distances in Table 7.4 show what the matching logic must report. The exact-match vector is at distance 0 from slot 0, and the approximate-match vector is at distance 1, so both fall within the threshold of 4 and are approved for slot 0. The non-match vector is at distance 48 from slot 0 and 80 from slot 1, which is far outside the threshold, so it is declined. These are the outcomes that the "APPROVED" and "DECLINED" messages in the testbench are written to report.

8.4 Hardware Implementation

Figure 8.2 shows the hardware setup. The Nexys A7 board, with its Artix-7 FPGA, is powered and programmed over USB. The optical fingerprint sensor is connected to the board with jumper wires, and the board's status LEDs are active. This is the setup on which the design was demonstrated.

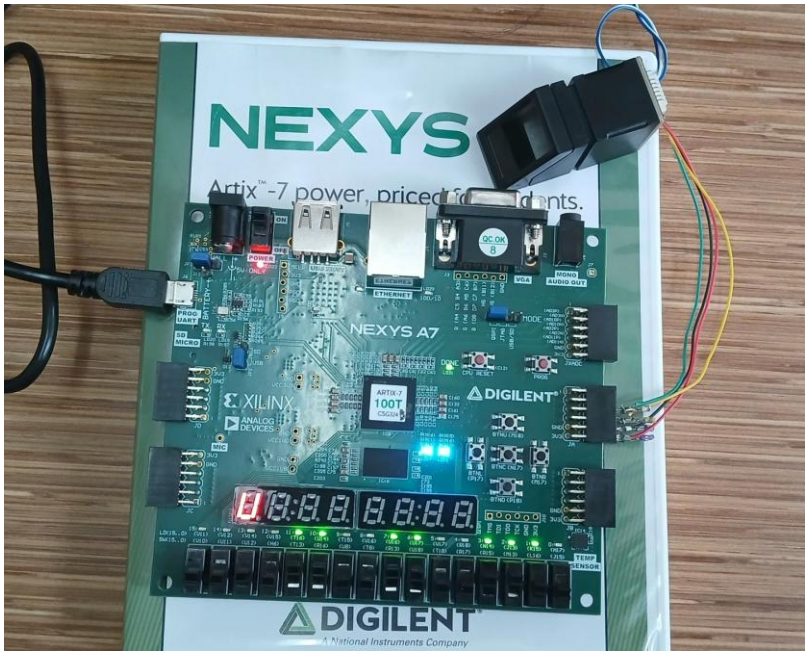


Figure 8.2: Hardware implementation: Nexys A7 (Artix-7) board with the optical fingerprint sensor.

8.5 Discussion

The simulation shows the behaviour we set out to build: a fingerprint request goes in, the decision is made by logic inside the FPGA, an approved transaction updates the balance, and a rejected one does not. Nothing in this path runs on a processor or an operating system, which is the main advantage of the FPGA approach described in Chapter 2.

It is worth being clear about what has and has not been measured. The figures of under one second per payment and a FAR below 0.001% quoted in Chapter 2 belong to the work of Rong and Liu [6], not to our prototype. We did not measure FAR or FRR, because that needs fingerprints from many people and many repeated scans. The only timing we give for our own core is the design-level estimate in Section 7.6. Finally, the waveform is a single run, so it shows that the logic behaves correctly for the cases in it; it is not a statistical result.

CHAPTER 9
ADVANTAGES, APPLICATIONS, LIMITATIONS AND FUTURE SCOPE

9.1 Advantages

- **Security.** The fingerprint is unique and constant, so it is much harder to forge, share or guess than a PIN or a card.
- **Convenience.** The user does not need to remember a PIN or carry a card.
- **Speed.** The matching decision is made in hardware in about a thousand clock cycles at most (Section 7.6).
- **Offline operation.** All the logic and the templates are inside the device, so no network connection is needed to authorise a payment.
- **Smaller attack surface.** Because the logic is in hardware, the software-level exploits that target an operating system do not apply in the same way.
- **Parallelism and scalability.** Independent blocks can work at the same time, and the number of slots and the template width are parameters of the design.

9.2 Applications

The design suits settings where connectivity is poor or cannot be guaranteed, and where a fast, self-contained terminal is useful:

- small retail shops and vending points;
- campus canteens, libraries and similar prepaid-account services;
- ticketing and access-controlled payments;
- offline POS terminals in areas with limited network coverage.

9.3 Limitations

- **Simplified template.** The prototype represents a fingerprint as a fixed 128-bit word and compares it by Hamming distance. A full minutiae matcher also has to cope with shifts and rotations of the finger. The listing in Appendix A covers storage and matching of the template; the step that produces the 128-bit word from the sensor output is not part of it.
- **No accuracy measurements.** FAR and FRR were not measured (Section 8.5).

- **No liveness detection.** A fake fingerprint presented to the sensor is not detected, so spoofing remains an open risk.
- **Template protection.** The templates are held in plain registers in the Verilog model. The encryption and security module of the proposed architecture is not part of the simulated code. The registers are also cleared by a reset, so a real product needs non-volatile storage.
- **Capacity.** The core holds eight templates, and the bit counter works serially at one bit per clock cycle. Both are suitable for a prototype but would be changed for a larger user base.
- **Synthesis clean-up.** The Verilog in Appendix A is written for simulation first. A few constructs, for example the `disable` statement noted in its own comment and some registers that are written from more than one `always` block, depend on the tool and should be tidied before a final synthesis.

9.4 Future Scope

The literature points to several directions, and we would add some of our own.

- **Deep learning.** Convolutional Neural Networks (CNNs) are a growing trend for stronger liveness detection and for improving matching on poor-quality or latent fingerprints.
- **Mobile biometrics.** As smartphones become the main way to pay, lightweight, energy-efficient and secure biometric algorithms designed for mobile platforms will be needed.
- **Decentralisation.** Blockchain could be used for decentralised management of user identity and transaction logs, adding security and transparency.
- **Security and accuracy.** The central challenge remains the trade-off between the two. A hardware-based design combined with a high-quality sensor module may be the best way to balance them.
- **Hardware security.** Adding an encryption core for stored templates, liveness detection, and non-volatile template storage.
- **Complete terminal.** Adding a VGA display and a keyboard interface, as in [6], and connecting the terminal to a bank or wallet back end for recharge and settlement.
- **Accuracy testing.** Collecting fingerprints from a group of volunteers to measure FAR and FRR for different thresholds.

CHAPTER 10
CONCLUSION

10.1 Summary

This report looked at the move from traditional payment methods to fingerprint-based biometric systems, driven by the need for better security and convenience. The main obstacles to wide use of the technology are security risks such as sensor spoofing and template theft, and keeping recognition accurate when the input is poor. Database-centric systems and software-based systems exist, but they either need a constant connection or bring in software vulnerabilities.

An FPGA-based design points in a promising direction. By implementing the authentication and transaction logic purely in hardware, such a system has the potential to be secure, fast, low in cost and able to work offline. The hardware-centric design bridges the gap between the security promise of biometrics and the practical demands of a quick, self-contained payment terminal.

In our own work, we designed and wrote in Verilog a template storage and matching core with a four-state finite state machine, verified it with a testbench and a waveform simulation, and demonstrated it on an Artix-7 FPGA board with an optical fingerprint sensor. The simulation showed an approved transaction updating the account balance and a rejected one leaving it unchanged. The prototype has clear limits, listed in Chapter 9, and we regard it as a proof of concept for a fast, reliable and fraud-resistant payment system that reduces unauthorised access.

10.2 Contributions of Team Members

Table 10.1: Contribution of each team member.

Team member	Contribution
Channayya V Meti	Code compilation and implementation
Jeevan K Hosakeri	Output execution and testing
Chetan Torgalmath	Research and data compilation
Narendar T Rathor	Code compilation and execution

REFERENCES

[1] Ogbanufe, O., & Kim, D. J. (2018). Comparing fingerprint-based biometrics authentication versus traditional authentication methods for e-payment. *Decision Support Systems*, 106, 1-14.

[2] Liébana-Cabanillas, F., Kalinic, Z., Muñoz-Leiva, F., & Higueras-Castillo, E. (2024). Biometric m-payment systems: A multi-analytical approach to determining use intention. *Information & Management*, 61(2024), 103907.

[3] Yang, W., Wang, S., Hu, J., Zheng, G., & Valli, C. (2019). Security and accuracy of fingerprint-based biometrics: A review. *Symmetry*, 11(2), 141.

[4] Nayak, A. S., Shetti, G. G., Shet, I., & Nayak, J. (2024). Payment transaction using biometric authentication. *Journal of Emerging Technologies and Innovative Research (JETIR)*, 11(4), d639-d645.

[5] Lavadkar, M. A., Thorat, P. K., Kasliwal, A. R., Gadekar, J. S., & Deshmukh, P. (n.d.). Fingerprint biometric based online cashless payment system. *IOSR Journal of Computer Engineering (IOSR-JCE)*, 27-32.

[6] Rong, B., & Liu, C. (2016). A fingerprint payment system based on FPGA. *2016 International Conference on Information System and Artificial Intelligence (ISAI)*, 472-475.

[7] Logeswari, R., Saikoteswara, M., Sudheerkumar, M. V., & Suryateja, J. S. R. K. (2018). Fingerprint based online payment system. *International Research Journal of Engineering and Technology (IRJET)*, 5(10), 573-579.

[8] Banumathi, K. L., Shreyas, P., Suhas, S., Vyshnavi, H. K., & Chitra, A. (2024). Biometric based payment system. *World Journal of Advanced Research and Reviews*, 23(01), 1880-1892.

APPENDIX A
VERILOG SOURCE CODE

The listing below is the `fingerprint_payment` module described in Chapter 7. It is reproduced from the project files.

```
module fingerprint_payment #(
    parameter TEMPLATE_W = 128,
    parameter SLOTS = 8,
    parameter THRESHOLD = 8)
    (input  wire                        clk,
     input  wire                        rst_n,

     // Enrollment interface
     input  wire                        enroll_valid,
     input  wire [TEMPLATE_W-1:0]       enroll_template,
     output reg                         enroll_ack,

     // Transaction interface
     input  wire                        tx_valid,
     input  wire [TEMPLATE_W-1:0]       tx_template,
     output reg                         tx_processing,
     output reg                         tx_approved,
     output reg                         tx_declined,
     output reg [$clog2(SLOTS)-1:0]     matched_slot
    );

    // Storage for templates (simple register file)
    reg [TEMPLATE_W-1:0] template_mem [0:SLOTS-1];
    reg [SLOTS-1:0]      slot_used; // flag per slot
    integer              i;

    // FSM states
    typedef enum logic [2:0] {
        IDLE,
        ENROLL_STORE,
        TX_SEARCH,
        TX_DONE
    } state_t;
    state_t state, next_state;

    // Internal search variables
    reg [$clog2(SLOTS+1)-1:0] search_idx;
    reg [TEMPLATE_W-1:0] xor_val;
    reg [$clog2(TEMPLATE_W+1)-1:0] hamming_count; // counts set bits in xor
    reg [$clog2(SLOTS+1)-1:0] best_match_idx;
    reg match_found;
    reg [$clog2(TEMPLATE_W+1)-1:0] pop_idx;
    reg pop_running;

    // reset / initialization
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            enroll_ack    <= 0;
            tx_processing <= 0;
            tx_approved   <= 0;
            tx_declined   <= 0;
            matched_slot  <= {($clog2(SLOTS)){1'b0}};
            slot_used     <= {SLOTS{1'b0}};
            for (i=0; i<SLOTS; i=i+1) template_mem[i] <= {TEMPLATE_W{1'b0}};
            state <= IDLE;
            search_idx <= 0;
            pop_idx <= 0;
            pop_running <= 0;
            match_found <= 0;
            best_match_idx <= 0;
        end else begin
            state <= next_state;
            // ack pulses: clear after one cycle
```

```
            if (enroll_ack) enroll_ack <= 0;
            if (tx_approved) tx_approved <= 0;
            if (tx_declined) tx_declined <= 0;
        end
    end

    // Next-state logic & main operations
    always @(*) begin
        // default next state
        next_state = state;
        case (state)
            IDLE: begin
                if (enroll_valid) next_state = ENROLL_STORE;
                else if (tx_valid) next_state = TX_SEARCH;
            end

            ENROLL_STORE: begin
                next_state = IDLE;
            end
            TX_SEARCH: begin
                if (match_found) next_state = TX_DONE;
                else if (search_idx == SLOTS) next_state = TX_DONE;
                else next_state = TX_SEARCH; // stay until finished
            end

            TX_DONE: begin
                next_state = IDLE;
            end

            default: next_state = IDLE;
        endcase
    end

    // Enrollment store logic
    always @(posedge clk) begin
        if (!rst_n) begin
            // handled in reset block above
        end else begin
            if (state == ENROLL_STORE) begin
                // find first free slot
                integer s;
                enroll_ack <= 0;
                for (s=0; s<SLOTS; s=s+1) begin
                    if (!slot_used[s]) begin
                        template_mem[s] <= enroll_template;
                        slot_used[s] <= 1'b1;
                        enroll_ack <= 1;
                        // end search after storing (non-synth friendly in some tools,
                        // but usually okay in Verilog simulation)
                        disable for;
                    end
                end
            end
        end
    end
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            tx_processing <= 0;
            search_idx <= 0;
            pop_idx <= 0;
            hamming_count <= 0;
            pop_running <= 0;
            match_found <= 0;
            best_match_idx <= 0;
            tx_declined <= 0;
            tx_approved <= 0;
        end else begin
            case (state)
                IDLE: begin
                    tx_processing <= 0;
                    search_idx <= 0;
                    pop_idx <= 0;
                    hamming_count <= 0;
                    pop_running <= 0;
                    match_found <= 0;
                    best_match_idx <= 0;
                end
```

```
                TX_SEARCH: begin
                    tx_processing <= 1;
                    // iterate until we examine all used slots
                    if (!pop_running) begin
                        // start popcount for current search_idx
                        if (search_idx < SLOTS) begin
                            if (slot_used[search_idx]) begin
                                xor_val <= template_mem[search_idx] ^ tx_template;
                                hamming_count <= 0;
                                pop_idx <= 0;
                                pop_running <= 1;
                            end else begin
                                // skip empty slot
                                search_idx <= search_idx + 1;
                            end
                        end else begin
                            // done scanning
                            search_idx <= search_idx; // remain
                        end
                    end else begin


                        // running popcount one bit each cycle
                        hamming_count <= hamming_count + xor_val[pop_idx];
                        pop_idx <= pop_idx + 1;
                        if (pop_idx + 1 == TEMPLATE_W) begin
                            // popcount done for this slot
                            pop_running <= 0;
                            // check threshold
                            if (hamming_count + xor_val[pop_idx] <= THRESHOLD) begin
                                match_found <= 1;
                                best_match_idx <= search_idx;
                            end else begin
                                // continue to next slot
                                search_idx <= search_idx + 1;
                            end
                        end
                    end
                end
                TX_DONE: begin
                    tx_processing <= 0;
                    if (match_found) begin
                        tx_approved <= 1;
                        matched_slot <= best_match_idx;
                    end else begin
                        tx_declined <= 1;
                        matched_slot <= {($clog2(SLOTS)){1'b0}};
                    end
                end


                default: begin
                end
            endcase
        end
    end

endmodule
```

APPENDIX B

TESTBENCH SOURCE CODE

The listing below is the testbench `tb_fingerprint_payment` described in Section 7.7.

```
// tb_fingerprint_payment.v
`timescale 1ns/1ps
module tb_fingerprint_payment;
    parameter TEMPLATE_W = 128;
    parameter SLOTS = 8;
    parameter THRESHOLD = 4;


    reg clk;
    reg rst_n;


    // IO
    reg enroll_valid;
    reg [TEMPLATE_W-1:0] enroll_template;


    reg tx_valid;
    reg [TEMPLATE_W-1:0] tx_template;


    wire enroll_ack;
    wire tx_processing;
    wire tx_approved;


    wire tx_declined;
    wire [$clog2(SLOTS)-1:0] matched_slot;


    fingerprint_payment #(
        .TEMPLATE_W(TEMPLATE_W),
        .SLOTS(SLOTS),
        .THRESHOLD(THRESHOLD)
    ) DUT (
        .clk(clk), .rst_n(rst_n),
        .enroll_valid(enroll_valid), .enroll_template(enroll_template),
        .enroll_ack(enroll_ack),
        .tx_valid(tx_valid), .tx_template(tx_template),
        .tx_processing(tx_processing),
        .tx_approved(tx_approved), .tx_declined(tx_declined),
        .matched_slot(matched_slot)
    );


    // clk
    initial clk = 0;
    always #5 clk = ~clk; // 100MHz-ish in sim scale


    initial begin
        rst_n = 0;
        enroll_valid = 0;
        enroll_template = 0;
        tx_valid = 0;
        tx_template = 0;
        #20;
        rst_n = 1;
        #20;


        $display("=== TEST: Enroll slot0 ===");
        // Enroll a template (slot 0)
        enroll_template = 128'hA5A5A5A5A5A5A5A5A5A5A5A5A5A5A5A5; // sample
        enroll_valid = 1;
        #10;
        enroll_valid = 0;
        #50;


        $display("Enroll ack? %0d", enroll_ack);


        $display("=== TEST: Enroll slot1 ===");
```

```
        enroll_template = 128'hFFFFFFFF00000000FFFFFFFF00000000; // sample 2
        enroll_valid = 1;
        #10;
        enroll_valid = 0;
        #50;


        $display("=== TEST: Transaction match (exact) ===");
        tx_template = 128'hA5A5A5A5A5A5A5A5A5A5A5A5A5A5A5A5; // exact match to slot0
        tx_valid = 1;
        #10;
        tx_valid = 0;
        // Wait until system finishes
        wait (!tx_processing);
        #10;
        if (tx_approved) $display("Transaction APPROVED, matched_slot=%0d", matched_slot);
        if (tx_declined) $display("Transaction DECLINED");


        $display("=== TEST: Transaction approximate match (few bit differences) ===");
        // flip a few bits within THRESHOLD
        tx_template = 128'hA5A5A5A5A5A5A5A5A5A5A5A5A5A5A5A4; // 1-bit diff
        tx_valid = 1;
        #10;
        tx_valid = 0;
        wait (!tx_processing);
        #10;
        if (tx_approved) $display("Transaction APPROVED (approx), matched_slot=%0d", matched_slot);
        if (tx_declined) $display("Transaction DECLINED");


        $display("=== TEST: Transaction non-match ===");
        tx_template = 128'h0123456789ABCDEF0123456789ABCDEF; // no match
        tx_valid = 1;
        #10;
        tx_valid = 0;
        wait (!tx_processing);
        #10;
        if (tx_approved) $display("Transaction APPROVED (unexpected), matched_slot=%0d",
matched_slot);
        if (tx_declined) $display("Transaction DECLINED (expected)");


        $display("=== SIM END ===");
        #50;
        $finish;
    end

endmodule
```